FICHAS DE TRABAJO

# UD01 — Fichas de trabajo

**Módulo:** Sistemas Operativos Monopuesto · 1º SMR
**Unidad:** UD01 — Fundamentos del sistema operativo (RA1)

Seis fichas, una por cada epígrafe de la unidad, para trabajar justo después de la explicación de cada uno. La idea no es que memorices una respuesta, sino que apliques el método que se ha explicado en clase — por eso cada ficha trae, como mucho, un ejemplo ya resuelto de referencia por cada tipo de ejercicio, y el resto lo resuelves tú. Trae a mano tus apuntes del apartado correspondiente.

**Estas fichas se trabajan en clase, en papel, sin móvil ni portátil**, justo después de la explicación del epígrafe correspondiente — no son para casa. Algunos ejercicios te van a pedir explicar en voz alta cómo has llegado a tu respuesta: lo que importa es que sepas reconstruir el proceso, no solo dar el resultado final.

**Excepción:** los ejercicios marcados con 💻 sí requieren un equipo encendido, porque piden comprobar algo real en tu propio sistema (no se pueden resolver copiando una respuesta de otro sitio, porque el resultado depende de tu equipo concreto). Si en ese momento no hay equipos disponibles en el aula, el profesor los planteará como tarea para la sesión siguiente.

## Ficha 1 — El sistema informático y el software

**Apartado relacionado:** 1 · **CE:** a

### Ejercicio 1 — Clasificación

Clasifica cada elemento como **hardware**, **software de base** o **software de aplicación**, y justifica tu respuesta en una frase.

1. Núcleo de Linux
2. Navegador web
3. Controlador (driver) de la tarjeta gráfica
4. Memoria RAM
5. Editor de código
6. Antivirus
7. Firmware de la BIOS/UEFI
8. Hoja de cálculo
9. Reproductor multimedia
10. Gestor de particiones

11. CPU (procesador)
12. Windows 11

**Ejemplo resuelto:** *Controlador de la tarjeta gráfica* → software de base. Justificación: permite que el sistema operativo se comunique con ese componente concreto; el usuario no lo usa directamente para una tarea.

## Ejercicio 2 — Niveles del sistema informático

Completa el esquema de niveles, de más cercano al hardware a más cercano al usuario, colocando estos cinco elementos en su nivel: *Usuario* · *Hardware* · *Software de aplicación* · *Software de base* · *Lenguajes/entornos de programación*.

```
Nivel 5 (el más externo): _____________________
Nivel 4: _____________________
Nivel 3: _____________________
Nivel 2: _____________________
Nivel 1 (el más interno): _____________________
```

## Ejercicio 3 — Verdadero o falso

Indica si cada afirmación es verdadera o falsa. Si es falsa, corrígela.

1. El sistema operativo es software de aplicación.
2. Los lenguajes de alto nivel se ejecutan directamente sobre el hardware, sin traducción.
3. El software de base existe para que las aplicaciones no tengan que hablar directamente con el hardware.
4. Un antivirus es siempre software de aplicación, sin discusión posible.
5. El lenguaje máquina es el único que la CPU entiende de verdad.

## Ejercicio 4 — Caso corto

Lee el siguiente escenario e identifica cada elemento subrayado, indicando en qué nivel del sistema informático se sitúa:

> "Enciendes el ordenador; el firmware UEFI arranca, después carga el sistema operativo, que a su vez ejecuta tu navegador, con el que abres una página escrita por alguien que programó en JavaScript."

## Ejercicio 5 — 💻 Software de base real en tu equipo

En Windows, abre el **Administrador de dispositivos** y despliega al menos tres categorías. Elige tres dispositivos y anota el nombre del controlador de cada uno (pestaña "Controlador" en sus propiedades). Si tu equipo tiene también Linux, compara con `lspci` o `lsusb` en una terminal.

¿Alguno de esos tres dispositivos dejaría de funcionar si le faltara su controlador, aunque el hardware siga físicamente conectado?

# Ficha 2 — Funciones y arquitectura del sistema operativo

**Apartado relacionado:** 2 · **CE:** c, d

## Ejercicio 1 — Relaciona

Une cada función del sistema operativo con lo que hace:

| Función | |
|---|---|
| A. Gestión de procesos | 1. Organiza y da acceso ordenado al almacenamiento |
| B. Gestión de memoria | 2. Reparte el tiempo de CPU entre programas |
| C. Gestión de archivos | 3. Media entre las aplicaciones y los periféricos |
| D. Gestión de entrada/salida | 4. Organiza la RAM para que las aplicaciones no se pisen |

## Ejercicio 2 — Arquitectura por capas

Coloca cada elemento en su capa correspondiente (núcleo, controladores, shell, aplicaciones): `bash` · controlador de impresora · Microsoft Word · gestión de procesos del kernel · GNOME · PowerShell.

**Ejemplo resuelto:** `bash` → shell (es una forma de darle órdenes al sistema, en texto).

## Ejercicio 3 — Verdadero o falso: interfaz gráfica y núcleo

1. Un servidor sin interfaz gráfica no tiene sistema operativo.
2. La interfaz gráfica es la versión gráfica de la capa de shell.
3. "Núcleo del sistema operativo" y "núcleo de la CPU" son lo mismo.
4. Un núcleo monolítico es una arquitectura anticuada frente al microkernel.
5. Windows NT usa un núcleo híbrido.

## Ejercicio 4 — Tipos de núcleo

Completa la tabla:

| Tipo | Cómo organiza los servicios | Ejemplo real |
|---|---|---|
| Monolítico | | |
| Microkernel | | |
| Híbrido | | |

## Ejercicio 5 — Paginación por demanda

Responde brevemente:

1. ¿Qué es una "página" en el contexto de la gestión de memoria?
2. ¿Qué hace el sistema operativo cuando necesita una página que no está en RAM?
3. ¿Cómo se llama, en Windows y en Linux respectivamente, el espacio en disco que se usa cuando falta RAM?

### Ejercicio 6 — Windows y Linux, capa a capa

Completa la tabla comparativa:

| Elemento | Windows | Linux |
|---|---|---|
| Núcleo | | |
| Shell en texto | | |
| Shell gráfico | | |
| Gestión de software | | |

### Ejercicio 7 — 💻 Memoria real de tu equipo

Abre el Administrador de tareas de Windows (`Ctrl+Shift+Esc`) → pestaña Rendimiento → Memoria (en Linux, `free -h` en una terminal). Anota: RAM total instalada, RAM en uso ahora mismo, y si aparece "memoria virtual"/"archivo de paginación" (Windows) o "swap" (Linux), con cuánto espacio.

Si cerraras ahora mismo todas las aplicaciones abiertas, ¿bajaría a cero el uso de RAM? ¿Por qué no?

### Ejercicio 8 — 💻 Explora capa a capa en tu propio equipo

Localiza en tu equipo, uno a uno, los cinco elementos de la tabla del ejercicio 6: núcleo (`winver` en Windows / `uname -r` en Linux), shell en texto (abre una terminal y ejecuta un comando), shell gráfico (tu propio escritorio), gestión de software (Store o `apt list --installed`) y controladores (Administrador de dispositivos).

¿En qué capa pasas más tiempo tú, en tu día a día como usuario?

## Ficha 3 — Procesos y sus estados

**Apartado relacionado:** 3 · **CE:** e

### Ejercicio 1 — Programa o proceso

Indica si cada situación describe un programa o un proceso:

1. El archivo `chrome.exe` guardado en el disco, sin abrir.
2. Tres pestañas de Chrome abiertas a la vez.
3. Un script de Python que nunca se ha ejecutado.
4. Ese mismo script mientras se está ejecutando.

## Ejercicio 2 — Secuencia de estados

Lee el escenario y escribe la secuencia de estados (nuevo, listo, ejecución, bloqueado, terminado) por la que pasa el proceso:

> "Abres un programa de edición de vídeo y le pides que exporte un archivo. El proceso empieza a procesar los fotogramas. En un momento dado, tiene que esperar a que el disco duro termine de escribir un archivo temporal. Cuando el disco responde, el proceso continúa y termina la exportación."

**Ejemplo resuelto (primer tramo):** al abrir el programa, el proceso pasa de *nuevo* a *listo*, y en cuanto la CPU le asigna tiempo, a *ejecución*.

## Ejercicio 3 — Verdadero o falso

1. Un proceso bloqueado está roto y hay que reiniciarlo.
2. Un proceso puede pasar varias veces por el estado "listo" a lo largo de su vida.
3. "Terminado" y "bloqueado" son el mismo estado.

## Ejercicio 4 — 💻 Observación real

**A)** Abre el Administrador de tareas (Windows) o ejecuta `ps aux` en una terminal Linux, y anota el nombre de 5 procesos que veas en ejecución en este momento. ¿Reconoces alguno como una aplicación que hayas abierto tú?

**B) Un programa, varios procesos.** Abre tu navegador y crea tres pestañas nuevas en páginas distintas. Cuenta en el Administrador de tareas (o con `ps aux | grep <navegador>` en Linux) cuántos procesos aparecen asociados a ese navegador. Si el navegador es un único programa instalado en tu disco, ¿por qué aparecen varios procesos?

# Ficha 4 — Sistema de archivos: organización y atributos

**Apartado relacionado:** 4 · **CE:** f, g

## Ejercicio 1 — Rutas

Dado este árbol de directorios de Linux:

```
/
├── home/
│   └── alumno/
│       ├── documentos/
│       │   └── memoria.pdf
│       └── practicas/
│           └── ud01/
│               └── ficha1.txt
├── etc/
└── var/
```

1. Escribe la ruta absoluta de `ficha1.txt`.
2. Escribe una ruta relativa a `ficha1.txt` suponiendo que partes de `/home/alumno`.
3. Escribe la ruta absoluta de `memoria.pdf`.
4. Escribe una ruta relativa a `memoria.pdf` suponiendo que partes de `/home/alumno/practicas`.

**5) 💻 Tu propia ruta real.** Abre el explorador de archivos de tu equipo Windows, navega hasta tu carpeta de Documentos y copia la ruta completa desde la barra de direcciones. Escríbela aquí como ruta absoluta, y después escribe una ruta relativa hasta esa misma carpeta partiendo de `C:\Users\<tu usuario>`.

### Ejercicio 2 — Mayúsculas y minúsculas

De estos nombres de archivo, ¿cuántos son archivos **distintos** en Windows? ¿Y en Linux?

`Memoria.pdf` · `memoria.pdf` · `MEMORIA.PDF` · `Memoria.PDF`

💻 **Compruébalo de verdad:** en Windows, crea `prueba.txt` y, en la misma carpeta, intenta crear `Prueba.txt`. ¿Qué pasa? Arranca el mismo equipo en Linux y repite el experimento con los mismos dos nombres. ¿Cuántos archivos distintos tienes ahora?

### Ejercicio 3 — Atributos

Completa la tabla:

| | Windows | Linux |
|---|---|---|
| Cómo se oculta un archivo | | |
| Otros atributos disponibles | | |

💻 **Compruébalo:** crea un archivo de prueba, márcalo como oculto (Windows: Propiedades → "Oculto"; Linux: renómbralo con un punto delante, `.prueba.txt`) y comprueba que desaparece del explorador con la vista de ocultos desactivada.

### Ejercicio 4 — Caso corto

Quieres esconder un archivo con información sensible marcándolo como oculto. ¿Es suficiente para protegerlo? Justifica tu respuesta con lo visto en clase (y con el ejercicio 3 si lo has hecho con equipo).

## Ficha 5 — Binario, codificación de caracteres y unidades de medida

**Apartado relacionado:** 5 · **CE:** b, h

## Ejercicio 1 — Batería de conversión numérica

Convierte cada valor según se indica. Trabaja siempre con un byte completo (8 bits) cuando el resultado sea binario. Recuerda: para octal se agrupa el binario de 3 en 3 bits (desde la derecha); para hexadecimal, de 4 en 4.

**A) Decimal → Binario**

1. 37
2. 91

**B) Binario → Decimal**

1. `10110101`
2. `11000011`

**C) Binario → Octal**

1. `10101010`
2. `11110000`

**D) Octal → Binario**

1. `62`
2. `17`

**E) Binario → Hexadecimal**

1. `10111001`
2. `01000110`

**F) Hexadecimal → Binario**

1. `9A`
2. `2D`

**G) Reto — conversión directa (sin pasos intermedios escritos, si te ves capaz)**

1. 156 (decimal) → hexadecimal
2. `3F` (hexadecimal) → decimal

**Ejemplo resuelto (apartado A):** 13 (decimal) → `00001101` (binario, 8 bits).

## Ejercicio 2 — ASCII

Usando la tabla ASCII reducida de tus apuntes, codifica la palabra `SO` letra a letra en valores decimales. Después, decodifica esta secuencia: `72 73`.

## Ejercicio 3 — UTF-8 (personalizado)

Elige una letra con tilde o la letra "ñ" que aparezca en tu nombre o en tu primer apellido. Si no tienes ninguna, usa la letra 'á'. Busca su *code point* en esta tabla y, siguiendo el método resuelto

en clase para 'ñ' (U+00F1 → `C3 B1`), codifícala en UTF-8 mostrando cada paso: rango del *code point*, número de bytes, binario en 11 bits, reparto en el patrón de bits, resultado en hexadecimal.

| Letra | *Code point* |
|---|---|
| á | U+00E1 |
| é | U+00E9 |
| í | U+00ED |
| ó | U+00F3 |
| ú | U+00FA |
| ñ | U+00F1 |

## Ejercicio 4 — Batería de unidades de medida

**A) Tabla de potencias.** Completa, de memoria y después revisando con tus apuntes:

| Prefijo | Símbolo SI | Potencia de 10 | Símbolo IEC | Potencia de 2 |
|---|---|---|---|---|
| kilo/kibi | | | | |
| mega/mebi | | | | |
| giga/gibi | | | | |
| tera/tebi | | | | |
| peta/pebi | | | | |

**B) Conversión dentro de la misma norma.**

1. Convierte 3,5 GB a MB (SI).
2. Convierte 2.500 MB a GB (SI).
3. Convierte 4 TiB a GiB (IEC).
4. Convierte 8.192 MiB a GiB (IEC).

**C) SI ↔️ IEC, con un caso real.**

1. Un disco se anuncia como de 128 GB. ¿A cuántos GiB equivale, aproximadamente? Muestra el cálculo.
2. Una tarjeta SD se anuncia como de 2 TB. ¿A cuántos TiB equivale, aproximadamente?

**D) Bits y bytes.**

1. Tu conexión a internet es de 300 Mbps. Como máximo teórico, ¿cuántos MB puedes descargar en un minuto?
2. Un archivo pesa 750 MB. Con una conexión de 100 Mbps, ¿cuánto tardarás en descargarlo, como mínimo (en segundos)?
3. ¿Qué diferencia hay entre "Gb" y "GB"? ¿Cuál de los dos usan los proveedores de internet para anunciar la velocidad de su servicio, y cuál usan los fabricantes de discos para anunciar su capacidad?

**E) Integrador — ordena de menor a mayor:**

1. `500 MB` · `2 GiB` · `0,001 TB` · `1.500.000 KB`

**F) 💻 Comprobación real.** En Windows, clic derecho sobre la unidad `C:` → Propiedades. Anota la capacidad total que muestra el sistema (en GB) y, si la ves, la capacidad exacta en bytes. Busca la capacidad anunciada por el fabricante (caja, factura o especificaciones del equipo). ¿Coinciden? Si no, calcula a qué se debe la diferencia.

**Ejemplo resuelto (apartado B):** 256 MB a GB (SI) → 256 ÷ 1000 = 0,256 GB.

## Ejercicio 5 — Permisos: de simbólico a octal

Convierte a notación octal:

1. `rwxr-xr-x`
2. `rw-rw-r--`
3. `rwx------`
4. `r--r--r--`
5. `rw-r-----`

## Ejercicio 6 — Permisos: de octal a simbólico

Convierte a notación simbólica y explica qué puede hacer cada tipo de usuario (propietario/grupo/otros):

1. `751`
2. `640`
3. `770`

# Ficha 6 — Sistemas transaccionales y tipos de sistemas de archivos

**Apartado relacionado:** 6 · **CE:** i

## Ejercicio 1 — Journaling

Explica con tus propias palabras la diferencia entre que un sistema de archivos tenga journaling y que se hagan copias de seguridad periódicas, y por qué hacen falta las dos cosas.

## Ejercicio 2 — Panorama de sistemas de archivos

Completa la tabla de memoria (repasa después con tus apuntes):

| | SO nativo | Journaling | Permisos Unix | Sensible a mayúsculas |
|---|---|---|---|---|
| FAT32 | | | | |
| NTFS | | | | |

| | SO nativo | Journaling | Permisos Unix | Sensible a mayúsculas |
|---|---|---|---|---|
| ext4 | | | | |
| HFS+/APFS | | | | |

### Ejercicio 3 — Elegir sistema de archivos

Para cada escenario, indica qué sistema de archivos elegirías y justifica tu respuesta:

1. Un pendrive que debe leerse en Windows, Mac y en la cámara de fotos de un compañero.
2. Un disco externo dedicado solo a copias de seguridad de una carpeta con scripts ejecutables de Linux.
3. El disco principal de un portátil que va a llevar Windows 11.

### Ejercicio 4 — 💻 Comprobación real: el sistema de archivos de un pendrive

Conecta a tu equipo una memoria USB o disco externo que ya tengas. En Windows, clic derecho sobre la unidad → Propiedades → "Sistema de archivos" (en Linux, `lsblk -f` en una terminal). Anota qué sistema de archivos tiene y compáralo con la tabla del ejercicio 2: ¿tiene sentido para el uso que le das a ese dispositivo?

## Relación con RA y CE

| Ficha | CE que trabaja |
|---|---|
| 1 | a |
| 2 | c, d |
| 3 | e |
| 4 | f, g |
| 5 | b, h |
| 6 | i |